

INTERMEDIATE PROGRAMMER

CSS & STYLING FRAMEWORKS

Responsive, Maintainable, Framework-Aware,
dan Project-Ready Interface Engineering

TECHNICAL LEARNING BOOK — V2

PT Neuronworks Indonesia

Program Pengembangan Kompetensi Programmer | 2026

Edisi 2.0 / Agustus 2026

Panduan Penggunaan Modul

Posisi modul: Modul ini memvalidasi cara programmer yang sudah aktif di project mengambil keputusan styling, menemukan akar masalah, dan menghasilkan UI yang responsif serta mudah dipelihara. Peserta diharapkan dapat menjelaskan alasan teknis, bukan sekadar memperlihatkan hasil visual.

Komponen	Ketentuan
Level	Intermediate Programmer
Durasi	1 minggu; estimasi 10–14 jam self-learning di luar mentoring dan project
Prasyarat	Menguasai HTML/CSS Junior; terbiasa memakai browser DevTools, Git, terminal, Node.js, dan npm
Perangkat	Browser modern, DevTools, editor/IDE, Node.js/npm, dan repository latihan
Metode	Pahami → prediksi → jalankan → inspeksi → refactor → jelaskan keputusan
Bukti belajar	Source code, screenshot lintas viewport, catatan debugging, checklist, dan technical reasoning

Cara belajar yang disarankan

- Mulai dari kebutuhan dan bukti masalah; jangan memilih framework hanya karena familiar.
- Prediksi cascade, ukuran, atau layout sebelum membuka DevTools.
- Ubah satu variabel pada satu waktu, lalu dokumentasikan hasil observasinya.
- Uji pada keyboard, beberapa viewport, serta kondisi konten pendek dan panjang.
- Refactor solusi yang bekerja menjadi kode yang konsisten, reusable, dan dapat dijelaskan.

Batas kedalaman

Modul tidak menuntut hafalan seluruh utility Tailwind atau komponen Bootstrap. Fokusnya adalah CSS reasoning, pemilihan alat, integrasi yang benar, kualitas implementasi, dan kemampuan membuktikan penyebab masalah. AI boleh membantu eksplorasi, tetapi bukan kompetensi wajib dan setiap output tetap harus diverifikasi peserta.

Peta Pembelajaran V2

CARA MEMBACA MODUL		
Gunakan jalur Core untuk kompetensi yang langsung merujuk silabus. Supporting menjelaskan praktik yang membuat implementasi Core aman dan maintainable. Reference dipakai saat sesuai kebutuhan project dan bukan target hafalan.		
Label	Makna	Perlakuan belajar
CORE	Langsung berasal dari silabus Intermediate.	Wajib dipahami, dipraktikkan, dan dapat dijelaskan.
SUPPORTING	Diperlukan agar Core aman dan benar.	Pelajari dalam konteks implementasi dan failure mode.
REFERENCE	Pendalaman atau variasi tool/framework.	Gunakan ketika relevan; tidak menjadi hafalan utama.

Learning Path Satu Minggu

Hari	Fokus	Bab	Evidence
1	CSS reasoning dan sizing	1–2	Prediksi, DevTools, token, baseline
2	Layout, layer, responsive	3–5	Matriks viewport dan debugging log
3	Component quality dan Tailwind	6–8	State matrix dan build evidence
4	Bootstrap dan framework decision	9–11	Regression test dan decision record
5	Mini-project dan technical defense	12	Evidence package dan peer review

Bukti minimum peserta

- Source code dan instruksi setup/build yang dapat direproduksi.
- Screenshot lintas viewport, zoom, state, serta data ekstrem.
- Satu debugging log lengkap dari gejala sampai regression check.
- Decision record pemilihan framework, layout, breakpoint, dan abstraction.
- Technical reasoning yang merujuk requirement, mekanisme, bukti, dan trade-off.

Ringkasan Traceability

Kelompok silabus	Bab	Bukti utama
CSS properties, selector, calculation, unit, z-index	1–4, 6	Computed style, token, layout/layer evidence
Responsive, media query, Flexbox, maintenance	3, 5, 11	Matriks viewport dan debugging log
Tailwind introduction dan styling	6–8	Build, variants, theme, component refactor
Bootstrap setup, customization, layout, content	9	Reproducible setup dan regression evidence
Bootstrap forms dan validation	10	State, accessibility, dan server contract
Integrated validation	12	Dashboard dan evidence package

Catatan normalisasi silabus

NORMALISASI
Topik Flexbox yang menduplikasi teks media query dipetakan ke Bab 3 dan 5 sesuai maknanya. Simbol unit CSS yang tertulis '&' pada sumber diperlakukan sebagai '%' berdasarkan konteks. Normalisasi tidak menghapus cakupan materi.

Capaian Pembelajaran

- mendiagnosis box model, cascade, inheritance, specificity, selector, dan computed style;
- memilih unit dan fungsi CSS yang adaptif terhadap konteks;
- membangun layout dengan Flexbox/Grid serta mengendalikan overflow;
- menjelaskan positioning, stacking context, dan z-index secara deterministik;
- menerapkan responsive design mobile-first berdasarkan kebutuhan konten;
- membangun komponen dengan state, token, motion, dan focus yang konsisten;
- mengimplementasikan Tailwind CSS dan Bootstrap tanpa kehilangan reasoning CSS;
- menelusuri konflik framework atau legacy CSS secara sistematis; dan
- menyelesaikan dashboard responsif dengan bukti pengujian dan technical reasoning.

Peta Kompetensi

Area	Kemampuan inti	Bukti kompetensi
CSS reasoning	Memprediksi cascade, ukuran, dan computed value	Hipotesis dan bukti DevTools selaras
Layout	Memilih flow, Flexbox, Grid, positioning, dan overflow	Layout stabil pada variasi konten
Responsive	Mobile-first dan breakpoint berbasis konten	Matriks uji beberapa viewport
Component quality	Token, state, focus, motion, dan reuse	Komponen konsisten dan accessible
Framework	Memakai Tailwind/Bootstrap secara terukur	Build benar; override dan trade-off terdokumentasi
Debugging	Reproduce, inspect, isolate, fix, retest	Log akar masalah dan regresi
Maintainability	Mengendalikan specificity, duplikasi, dan coupling	Diff refactoring dan keputusan teknis

Komposisi Pembelajaran

Aktivitas	Proporsi	Fokus
Konsep	25%	Mental model dan prinsip
Implementasi	35%	Eksperimen dan penerapan
Debugging/refactoring	25%	Akar masalah dan kualitas kode
Mini-project/reasoning	15%	Bukti kompetensi terintegrasi

Isi Buku

- 01** Mendiagnosis fondasi CSS
 - 02** Unit adaptif dan kalkulasi
 - 03** Layout engineering dengan Flexbox dan Grid
 - 04** Positioning, stacking context, dan overlay
 - 05** Responsive design berbasis konten
 - 06** Kualitas komponen, state, dan motion
 - 07** Tailwind CSS: workflow inti
 - 08** Tailwind CSS: customization dan maintainability
 - 09** Bootstrap: setup, grid, dan content system
 - 10** Bootstrap: forms, validation, dan customization
 - 11** Debugging dan keputusan framework
 - 12** Implementasi terpadu dan mini-project
-

BAB 01

Mendiagnosis Fondasi CSS

Menjelaskan mengapa sebuah declaration menang, bagaimana ukuran aktual terbentuk, dan bagaimana selector memengaruhi coupling.

Klasifikasi	CORE
Fokus silabus	CSS Properties — margin/padding, border, combinator, pseudo-class/pseudo-element, attribute selector, dan dasar properties.
Project case	Card melebar di production dan warna tombol hanya dapat diperbaiki dengan !important. Peserta harus membuktikan ukuran aktual serta rule pemenang melalui DevTools.
Bukti Intermediate	Prediksi box model, screenshot Computed/Styles, selector hasil refactor, dan regression check.
Best practice	Isolasi penyebab cascade atau ukuran terlebih dahulu; hindari menaikkan specificity sebagai respons otomatis.

TARGET KOMPETENSI

- Membuktikan box model dan computed value melalui DevTools.
- Menentukan pemenang cascade tanpa menambah !important.
- Memilih selector yang cukup spesifik, tetapi tetap mudah diubah.

1.1 Box model dan ukuran aktual

Pada content-box, width hanya mengatur area konten; padding dan border menambah ukuran luar. Pada border-box, width mencakup content, padding, dan border. Margin berada di luar box dan dapat mengalami margin collapsing pada alur blok tertentu.

Contoh 1. Baseline box sizing

```
*, ::before, ::after {
  box-sizing: border-box;
}

.card {
  width: 20rem;
  padding: 1.25rem;
  border: 1px solid #cbd5e1;
  margin-block: 1rem;
}
```

Gejala	Hipotesis awal	Bukti
Card lebih lebar dari container	content-box atau min-width child	Computed width dan overlay box model
Jarak vertikal tidak terjumlah	margin collapsing	Bandingkan margin parent/first child
Input keluar dari panel	width 100% + padding pada content-box	Aktifkan border-box dan bandingkan

1.2 Cascade, inheritance, dan specificity

Keputusan style dipengaruhi origin dan importance, cascade layer, specificity, proximity, lalu inheritance. Specificity bukan skor yang sebaiknya terus dinaikkan; ia adalah konsekuensi selector. Gunakan scope dan arsitektur yang jelas agar override tetap terkendali.

Contoh 2. Urutan layer yang eksplisit

```
@layer reset, base, components, utilities;

@layer base {
  :root { color: #0f172a; }
  button { font: inherit; }
}

@layer components {
  .button { background: #2563eb; }
  .button[aria-disabled="true"] { opacity: .55; }
}
```

Red flag: Jika perbaikan pertama selalu menambahkan selector lebih panjang atau !important, kemungkinan masalah sebenarnya adalah source order, scope, state, atau layer yang belum dirancang.

1.3 Selector dinamis

Jenis	Contoh	Penggunaan
Combinator	.toolbar > button	Relasi struktur langsung
Attribute	[data-state='open']	State dari DOM tanpa class tambahan
Pseudo-class	:focus-visible	Interaksi dan kondisi

Jenis	Contoh	Penggunaan
Pseudo-element	::before	Dekorasi yang tidak menambah makna
Low specificity	:where(.stack > *)	Rule dasar mudah dioverride

Contoh 3. Selector berbasis kondisi

```
.field:has(input:invalid) .field__hint { color: #b91c1c; }
.nav__link:is(:hover, :focus-visible) { text-decoration: underline; }
.stack > * + * { margin-block-start: 1rem; }
.badge::before { content: ""; inline-size: .5rem; block-size: .5rem; }
```

RED FLAG — KESALAHAN UMUM

- Menyamakan margin dan padding.
- Menganggap rule terakhir selalu menang tanpa melihat specificity/layer.
- Mengandalkan selector DOM yang panjang dan rapuh.
- Menggunakan pseudo-element untuk informasi yang harus dibaca assistive technology.

PRAKTIK & EVIDENCE 1

Debug card yang melebar dan tombol yang warnanya tidak dapat dioverride. Sebelum mengubah kode, tulis prediksi ukuran luar dan rule pemenang; buktikan dengan Computed dan Styles.

Definition of done: Tidak ada !important baru; ukuran card sesuai container; rule pemenang dapat dijelaskan; selector hasil refactor maksimal dua class/attribute; bukti sebelum–sesudah tersimpan.

CHECKPOINT Bab 1

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa yang berubah ketika box-sizing menjadi border-box?

Mengapa specificity tinggi meningkatkan biaya perubahan?

Kapan attribute selector lebih tepat daripada class state?

BAB 02

Unit Adaptif dan Kalkulasi

Membangun ukuran yang proporsional terhadap font, container, dan viewport tanpa mengorbankan batas keterbacaan.

Klasifikasi	CORE
Fokus silabus	CSS Properties — CSS units, counter, dan calculation.
Project case	Halaman terlihat baik di desktop tetapi overflow pada mobile karena ukuran tetap, spacing acak, dan typography tanpa batas.
Bukti Intermediate	Token CSS, penggunaan clamp/min/max/calc, serta screenshot 320 px sampai desktop lebar.
Best practice	Pilih unit berdasarkan referensi ukuran yang harus dipertahankan dan selalu tetapkan batas minimum–maksimum.

TARGET KOMPETENSI

- Memilih px, %, em, rem, viewport unit, dan container-relative sizing sesuai konteks.
- Menggunakan calc(), min(), max(), dan clamp() untuk batas adaptif.
- Menggunakan custom property sebagai token, bukan angka tersebar.

2.1 Memilih unit

Unit	Relatif terhadap	Pilihan umum
px	CSS pixel	Border tipis, detail yang perlu stabil
%	Containing block/property terkait	Lebar fluida
em	Font-size elemen	Komponen yang ikut skala lokal
rem	Font-size root	Typography dan spacing konsisten

Unit	Relatif terhadap	Pilihan umum
vw/vh	Viewport	Area visual; perlu batas dan perhatian mobile viewport

Prinsip: Tidak ada satu unit terbaik untuk semua kasus. Pilihan yang baik mengikuti referensi ukuran yang ingin dipertahankan saat konteks berubah.

2.2 Fungsi matematika dan token

Contoh 4. Ukuran fluida dengan batas

```
:root {
  --space-2: .5rem;
  --space-4: 1rem;
  --content-max: 72rem;
  --title-size: clamp(1.75rem, 1.2rem + 2vw, 3rem);
}

.shell { width: min(100% - 2rem, var(--content-max)); margin-inline: auto; }
.page-title { font-size: var(--title-size); }
.sidebar { inline-size: clamp(15rem, 25vw, 22rem); }
.main { min-inline-size: 0; }
```

2.3 Counter untuk penomoran presentasional

Contoh 5. CSS counter

```
.steps { counter-reset: step; }
.steps > li { counter-increment: step; }
.steps > li::marker { content: "Langkah " counter(step) ". "; }
```

Counter membantu presentasi urutan, tetapi struktur list HTML tetap harus benar. Jangan memakai generated content sebagai satu-satunya sumber informasi penting.

PRAKTIK & EVIDENCE 2

Refactor halaman dengan font 32px, lebar 1200px, dan spacing acak. Buat token minimal, title fluida, dan shell dengan batas baca.

Definition of done: Tidak overflow pada 320px; title memiliki batas; empat nilai berulang menjadi token; keputusan unit dapat dijelaskan.

CHECKPOINT Bab 2

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Kapan em lebih berguna daripada rem?

Apa manfaat batas minimum dan maksimum pada clamp()?

Mengapa min-inline-size: 0 sering penting pada child Flex/Grid?

BAB 03

Layout Engineering dengan Flexbox dan Grid

Memilih model layout dari hubungan antarelemen, bukan dari kebiasaan atau trial-and-error.

Klasifikasi	CORE + SUPPORTING
Fokus silabus	CSS Properties/Flexboxes — layout, alignment, overflow, Flexbox; Grid digunakan sebagai penguat layout dua dimensi.
Project case	Toolbar pecah, metric card tidak sejajar, dan tabel memaksa halaman melebar ketika label serta data bertambah.
Bukti Intermediate	Perbandingan Flex/Grid, uji konten ekstrem, dan bukti tidak ada accidental overflow.
Best practice	Mulai dari normal flow, lalu pilih Flexbox untuk satu axis dan Grid untuk dua axis berdasarkan hubungan konten.

TARGET KOMPETENSI

- Membedakan normal flow, one-dimensional layout, dan two-dimensional layout.
- Menjelaskan main/cross axis, wrapping, grow, shrink, dan basis.
- Membangun grid responsif dengan minmax() dan auto-fit.

3.1 Normal flow, display, dan overflow

Normal flow adalah baseline yang paling tahan perubahan konten. Ubah display hanya ketika hubungan layout memerlukannya. Overflow bukan selalu kesalahan; tentukan apakah konten harus membungkus, dipotong, di-scroll, atau memengaruhi ukuran container.

Contoh 6. Flex layout yang mengizinkan shrink

```
.workspace {
  display: flex;
  gap: 1rem;
  align-items: flex-start;
}
.workspace__main { flex: 1 1 40rem; min-inline-size: 0; }
.workspace__aside { flex: 0 1 20rem; }
.table-region { overflow-x: auto; }
```

3.2 Flexbox

Properti	Pertanyaan yang dijawab
flex-direction	Ke arah mana main axis berjalan?
flex-wrap	Apakah item boleh pindah baris?
justify-content	Bagaimana ruang pada main axis dibagi?
align-items	Bagaimana item disejajarkan pada cross axis?
flex	Bagaimana basis, grow, dan shrink item?

3.3 Grid

Contoh 7. Grid adaptif

```
.metric-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
  gap: 1rem;
}

.dashboard {
  display: grid;
  grid-template-columns: minmax(0, 2fr) minmax(16rem, 1fr);
  gap: 1.25rem;
}
```

Gunakan	Ketika
Normal flow	Urutan dokumen sudah cukup dan harus mengikuti konten
Flexbox	Distribusi/alignment dominan pada satu axis
Grid	Baris dan kolom perlu dikendalikan bersama
Positioning	Elemen perlu keluar dari flow untuk hubungan khusus

RED FLAG — KESALAHAN UMUM

- Menyelesaikan semua layout dengan position:absolute.
- Memberi width tetap pada child tanpa mempertimbangkan shrink.
- Menggunakan justify-content untuk cross axis.
- Menambah breakpoint untuk masalah yang sebenarnya dapat diselesaikan auto-fit/minmax.

PRAKTIK & EVIDENCE 3

Bangun area filter + hasil + ringkasan. Gunakan Flexbox untuk toolbar dan Grid untuk metric cards. Uji label panjang, data kosong, dan tabel lebar.

Definition of done: Tidak ada overlap; toolbar dapat wrap; metric cards beradaptasi tanpa breakpoint tambahan; tabel tetap dapat diakses melalui scroll region; peserta menjelaskan pilihan Flex vs Grid.

CHECKPOINT Bab 3

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa perbedaan masalah satu axis dan dua axis?

Mengapa minmax(0, 1fr) dapat mencegah overflow?

Apa dampak flex-shrink: 0 pada ruang sempit?

BAB 04

Positioning, Stacking Context, dan Overlay

Mengendalikan elemen yang keluar dari flow tanpa perang z-index.

Klasifikasi	CORE + SUPPORTING
Fokus silabus	CSS Properties — penentuan z-index dan positioning yang mendasarinya.
Project case	Dropdown berada di belakang card meski memakai z-index besar, sedangkan sticky header tidak bekerja di scroll container.
Bukti Intermediate	Peta ancestor/stacking context, layer token, dan hasil uji overlay serta scroll.
Best practice	Temukan stacking context dan containing block sebelum mengubah angka z-index.

TARGET KOMPETENSI

- Memilih relative, absolute, fixed, atau sticky sesuai containing block dan perilaku scroll.
- Mengenali pembentuk stacking context.
- Mendesain layer token untuk dropdown, sticky region, backdrop, modal, dan toast.

4.1 Position dan containing block

Nilai	Perilaku utama	Contoh
relative	Tetap dalam flow; menjadi acuan child absolute	Badge di dalam card
absolute	Keluar dari flow; diposisikan terhadap containing block	Icon dekoratif
fixed	Umumnya terhadap viewport	Toast global
sticky	Berperilaku flow lalu menempel dalam scroll container	Header tabel

Contoh 8. Layer token

```

:root {
  --layer-dropdown: 20;
  --layer-sticky: 30;
  --layer-backdrop: 80;
  --layer-modal: 90;
  --layer-toast: 100;
}
.dialog-backdrop { position: fixed; inset: 0; z-index: var(--layer-backdrop); }
.dialog { position: fixed; inset: 50% auto auto 50%; z-index: var(--layer-modal); }
.table-head { position: sticky; top: 0; z-index: var(--layer-sticky); }

```

4.2 Stacking context

z-index dibandingkan di dalam stacking context yang sama. Sebuah child dengan z-index sangat besar tetap tidak dapat melampaui sibling dari ancestor yang berada pada stacking context lebih tinggi. Position dengan z-index, transform, opacity tertentu, isolation, dan beberapa properti lain dapat membentuk konteks baru.

Debugging: Saat overlay tertutup, telusuri ancestor yang membentuk stacking context. Jangan langsung menaikkan z-index ke 999999.

4.3 Sticky yang tampak tidak bekerja

- Pastikan inset seperti top ditetapkan.
- Identifikasi scroll container sebenarnya.
- Periksa overflow pada ancestor.
- Pastikan tersedia ruang scroll dan elemen tidak setinggi containernya.
- Uji apakah transform atau containment mengubah konteks.

PRAKTIK & EVIDENCE 4

Perbaiki dropdown yang berada di belakang card dan sticky header yang tidak menempel. Buat peta ancestor serta stacking context sebelum mengubah z-index.

Definition of done: Penyebab didokumentasikan; layer memakai token; dropdown/modal terlihat pada konteks tepat; keyboard focus tidak terperangkap di balik overlay; regresi scroll diuji.

CHECKPOINT Bab 4

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa z-index besar belum tentu menang?

Apa hubungan absolute dengan containing block?

Sebutkan dua pemeriksaan saat sticky tidak bekerja.

BAB 05

Responsive Design Berbasis Konten

Membangun pengalaman mobile-first yang berubah ketika konten membutuhkan ruang, bukan karena meniru daftar perangkat.

Klasifikasi	CORE
Fokus silabus	CSS Responsive serta Responsive Design — media queries, breakpoint, dan responsive maintenance.
Project case	Dashboard hanya diuji pada satu ukuran mobile dan desktop; perubahan kecil pada konten menyebabkan overflow serta action hilang.
Bukti Intermediate	Matriks viewport, uji zoom 200%, titik breakpoint, state panjang/kosong/error, dan debugging log.
Best practice	Tentukan breakpoint ketika konten rusak, bukan berdasarkan nama atau merek perangkat.

TARGET KOMPETENSI

- Menyusun base style untuk viewport sempit lalu menambah enhancement.
- Menentukan breakpoint dari titik rusaknya konten.
- Menguji overflow, typography, gambar, form, dan tabel lintas viewport.

5.1 Mobile-first

Contoh 9. Enhancement pada ruang yang cukup

```
.dashboard { display: grid; gap: 1rem; }
.toolbar { display: flex; flex-wrap: wrap; gap: .75rem; }
.toolbar > * { flex: 1 1 12rem; }

@media (min-width: 48rem) {
  .dashboard { grid-template-columns: minmax(0, 2fr) minmax(16rem, 1fr); }
  .toolbar > * { flex: 0 1 auto; }
}
```

5.2 Breakpoint dan matriks uji

Pilih breakpoint saat komponen tidak lagi terbaca, touch target saling berhimpitan, atau hierarki visual runtuh. Nilai breakpoint harus berasal dari kebutuhan layout, lalu diuji pada ukuran di bawah, tepat, dan di atas batas.

Viewport/kondisi	Yang diuji	Lulus jika
320–375px	Navigation, form, card, tabel	Tidak ada overflow halaman; aksi utama terjangkau
Sekitar breakpoint	Wrap dan perubahan grid	Tidak ada loncatan/ruang janggal
Tablet	Dua kolom dan kepadatan	Konten tidak terlalu sempit
Desktop lebar	Max-width dan line length	Konten tidak melebar tanpa batas
Zoom 200%	Reflow dan focus	Tidak ada informasi/aksi hilang

5.3 Media dan data lebar

Contoh 10. Guardrail responsif

```
img, svg, video { max-inline-size: 100%; block-size: auto; }
.data-table-wrap { overflow-x: auto; overscroll-behavior-inline: contain; }
@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after { scroll-behavior: auto; }
}
```

RED FLAG — KESALAHAN UMUM

- Menggunakan breakpoint berdasarkan merek perangkat.
- Menyembunyikan data penting pada mobile tanpa alternatif.
- Memasang overflow-x:hidden pada body untuk menutupi akar masalah.
- Hanya menguji satu ukuran desktop dan satu ukuran mobile.

PRAKTIK & EVIDENCE 5

Audit halaman project nyata pada 320px, 768px, 1024px, desktop lebar, dan zoom 200%. Catat titik kerusakan sebelum menambah media query.

Definition of done: Tidak ada accidental horizontal overflow; perubahan layout memiliki alasan konten; focus terlihat; tabel punya strategi; hasil uji sebelum/tepat/setelah breakpoint terdokumentasi.

CHECKPOINT Bab 5

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa arti mobile-first pada source order CSS?

Mengapa breakpoint sebaiknya berasal dari konten?

Mengapa overflow-x:hidden bukan perbaikan utama?

BAB 06

Kualitas Komponen, State, dan Motion

Menyatukan token, visual state, focus, dan motion menjadi komponen yang konsisten serta dapat digunakan kembali.

Klasifikasi	SUPPORTING
Fokus silabus	CSS Properties/Styling Tailwind — border, shadow, component state, transition, animation, dan reusability.
Project case	Komponen memiliki warna, spacing, hover, focus, dan motion yang berbeda-beda sehingga sulit dipelihara dan tidak accessible.
Bukti Intermediate	Token/state matrix, uji keyboard/focus, reduced motion, dan diff refactoring.
Best practice	Gunakan token untuk keputusan berulang dan definisikan state sebagai kontrak komponen, bukan styling kebetulan.

TARGET KOMPETENSI

- Membedakan token desain dari nilai lokal.
- Menyediakan state hover, focus, active, disabled, loading, success, dan error yang relevan.
- Menggunakan transition/animation tanpa mengganggu aksesibilitas atau performa.

6.1 Token dan kontrak komponen

Contoh 11. Token dan state tombol

```
:root {
  --color-primary: #1d4ed8;
  --color-danger: #b91c1c;
  --radius-md: .625rem;
  --shadow-card: 0 .5rem 1.5rem rgb(15 23 42 / .10);
  --duration-fast: 160ms;
}
.button {
  border-radius: var(--radius-md);
  transition: background-color var(--duration-fast), transform var(--duration-fast);
}
.button:focus-visible { outline: 3px solid #93c5fd; outline-offset: 2px; }
.button:active { transform: translateY(1px); }
.button:disabled, .button[aria-disabled="true"] { opacity: .55; cursor: not-allowed; }
```

6.2 Transition versus animation

Teknik	Gunakan untuk	Hindari
transition	Perubahan antara dua state	transition: all
animation	Urutan keyframe atau aktivitas berulang	Motion tanpa tujuan
transform/opacit y	Motion ringan yang sering lebih efisien	Menganggap selalu gratis

Contoh 12. Reduced motion

```
@media (prefers-reduced-motion: reduce) {  
  .button, .skeleton, .toast {  
    animation-duration: .01ms;  
    animation-iteration-count: 1;  
    transition-duration: .01ms;  
  }  
}
```

6.3 Review aksesibilitas visual

- ☐ Focus keyboard terlihat dan tidak tertutup sticky layer.
- ☐ State error tidak disampaikan dengan warna saja.
- ☐ Disabled berbeda dari read-only dan tidak dipakai sebagai pengganti otorisasi.
- ☐ Touch target serta jarak antaraksi cukup.
- ☐ Motion memiliki tujuan dan reduced-motion dihormati.

PRAKTIK & EVIDENCE 6

Refactor button, input, badge, dan alert dari halaman project menjadi komponen dengan token serta state yang konsisten. Uji keyboard dan reduced motion.

Definition of done: Nilai berulang menjadi token; focus-visible tersedia; error memiliki teks/icon; transition hanya pada properti perlu; tidak ada motion wajib untuk memahami informasi.

CHECKPOINT Bab 6

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa perbedaan token global dan nilai lokal?

Mengapa transition: all berisiko?

Apa yang harus terjadi ketika pengguna memilih reduced motion?

BAB 07

Tailwind CSS: Workflow Inti

Menggunakan utility-first secara sadar, dari pipeline build hingga responsive variants dan state.

Klasifikasi	CORE
Fokus silabus	Tailwind Introduction/Styling — core concepts, hover/state, responsive design, reuse, Grid/Flexbox, dan build workflow.
Project case	Class terlihat di source tetapi style hilang pada build production karena pipeline, import, atau class dinamis tidak terdeteksi.
Bukti Intermediate	Build reproducible, output CSS, responsive/state variants, serta reasoning properti di balik utility.
Best practice	Perlakukan utility sebagai CSS yang tetap tunduk pada cascade, layout, state, dan build pipeline.

TARGET KOMPETENSI

- Menyiapkan workflow Tailwind CLI atau PostCSS yang dapat direproduksi.
- Menerjemahkan layout dan state menjadi utility dengan reasoning CSS.
- Memastikan output production hanya memuat style yang diperlukan oleh project.

7.1 Setup CLI saat ini

Contoh 13. Tailwind CLI

```
npm install tailwindcss @tailwindcss/cli

/* src/input.css */
@import "tailwindcss";

# development build
npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch
```

Catatan versi: Ikuti dokumentasi versi yang terpasang pada project. Jangan menyalin setup versi lama tanpa memeriksa package dan cara import yang sedang digunakan.

7.2 Utility, state, dan responsive variant

Contoh 14. Card responsif

```
<section class="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
  <article class="rounded-xl border border-slate-200 bg-white p-5 shadow-sm
    transition-shadow hover:shadow-md focus-within:ring-2
    focus-within:ring-blue-500">
    <p class="text-sm text-slate-600">Project aktif</p>
    <p class="mt-2 text-3xl font-semibold text-slate-900">18</p>
  </article>
</section>
```

7.3 Reasoning di balik utility

Utility	CSS mental model
grid gap-4	display:grid dan gap token
md:grid-cols-2	Enhancement pada min-width breakpoint md
focus-within:ring-2	State container ketika child memiliki focus
xl:grid-cols-4	Empat track saat ruang memadai

7.4 Anti-pattern

- Menyalin ratusan utility tanpa memahami hasil computed CSS.
- Membentuk semua nilai sebagai arbitrary value sehingga token hilang.
- Menggabungkan class secara dinamis dengan cara yang tidak dapat dideteksi build.
- Mengabaikan semantic HTML karena fokus pada class.

PRAKTIK & EVIDENCE 7

Implementasikan metric cards dan toolbar menggunakan Tailwind. Catat tiga utility penting beserta CSS mental model dan alasan breakpoint-nya.

Definition of done: Build dapat diulang dari package script; tidak ada class yang gagal muncul; semantic HTML tetap benar; layout lolos matriks responsive; reasoning utility terdokumentasi.

CHECKPOINT Bab 7

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa fungsi `@import "tailwindcss"` pada workflow saat ini?

Mengapa utility-first tidak menghapus kebutuhan memahami CSS?

Apa risiko class name yang dibentuk dinamis?

BAB 08

Tailwind CSS: Customization dan Maintainability

Menghubungkan token project dengan utility, mengelola reuse, dan menambahkan CSS kustom hanya ketika memberi nilai.

Klasifikasi	CORE
Fokus silabus	Tailwind Introduction — custom styles, functions/directives, reuse, dan code maintenance.
Project case	Utility menjadi rangkaian panjang, nilai desain tersebar, dan custom class tidak mengikuti theme sehingga perubahan global mahal.
Bukti Intermediate	Theme variables, component extraction terukur, build debugging, dan decision record.
Best practice	Ekstrak pola hanya ketika stabil atau berulang; jangan memindahkan semua utility menjadi abstraksi prematur.

TARGET KOMPETENSI

- Menggunakan theme variables untuk token project.
- Menentukan kapan markup utility, component extraction, atau custom CSS lebih tepat.
- Mendiagnosis output build dan konflik style.

8.1 Theme variables

Contoh 15. Token dan layer komponen

```
@import "tailwindcss";

@theme {
  --color-brand-500: oklch(0.62 0.20 255);
  --color-brand-700: oklch(0.48 0.18 255);
  --radius-panel: 0.875rem;
}

@layer components {
  .project-panel { @apply rounded-[var(--radius-panel)] border bg-white p-5; }
}
```

Pertimbangan: Ekstraksi class komponen bukan tujuan otomatis. Lakukan saat pola memang berulang, memiliki kontrak yang stabil, dan penamaannya membantu pembaca—bukan sekadar memindahkan utility dari HTML.

8.2 Custom styles, functions, dan directives

Gunakan mekanisme resmi versi project untuk theme, custom utility, source registration, dan directive. Dokumentasikan alasan ketika custom CSS dipakai karena komponen kompleks, integrasi third-party, atau selector yang tidak nyaman diekspresikan sebagai utility.

Pilihan	Kekuatan	Risiko
Utility di markup	Konteks lokal terlihat; perubahan cepat	Markup padat
Komponen/template	Reuse perilaku dan struktur	Abstraksi prematur
@layer components	Selector kompleks dan style reusable	Cascade baru perlu dikelola
Arbitrary value	Kasus sangat spesifik	Token drift bila terlalu sering

8.3 Debugging build

Konfirmasi file input dan output yang dimuat browser.

Periksa versi package dan command build.

Cari class literal pada source; hindari konstruksi nama yang tak terdeteksi.

Inspeksi rule pada Styles/Computed.

Pastikan custom layer/import berada pada urutan yang dimaksud.

Jalankan production build dan uji ulang halaman penting.

PRAKTIK & EVIDENCE 8

Refactor satu layar Tailwind yang banyak arbitrary value dan duplikasi. Bentuk token/theme, ekstrak hanya pola stabil, lalu ukur perubahan source dan output.

Definition of done: Arbitrary value berulang berkurang; token punya nama bermakna; state dan breakpoint tetap bekerja; build development/production lolos; refactor disertai trade-off.

CHECKPOINT Bab 8

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Kapan @layer components layak digunakan?

Mengapa arbitrary value berulang menandakan token drift?

Apa langkah pertama ketika utility tidak muncul di browser?

BAB 09

Bootstrap: Setup, Grid, dan Content System

Memanfaatkan sistem Bootstrap secara konsisten tanpa menumpuk override yang sulit diprediksi.

Klasifikasi	CORE
Fokus silabus	Bootstrap Introduction/Customization/Layout/Content — CDN, package/module, structure, global styles, grid, utilities, Reboot, typography, image, table, dan figures.
Project case	Bootstrap ditambahkan ke aplikasi legacy dan mengubah typography, spacing, serta layout global tanpa disadari.
Bukti Intermediate	Setup/build record, audit Reboot, grid responsif, dan uji regression content.
Best practice	Tentukan ownership CSS dan dampak baseline global sebelum mengintegrasikan framework.

TARGET KOMPETENSI

- Memilih CDN, package, atau module sesuai pipeline project.
- Menggunakan container, grid 12 kolom, breakpoint, gutter, dan utilities.
- Memahami Reboot, typography, images, tables, dan figures sebagai bagian sistem global.

9.1 Setup dan ownership

Cara	Cocok untuk	Kontrol
CDN	Prototype atau halaman sederhana	Cepat, customization terbatas
Package/module	Aplikasi dengan bundler	Import terukur dan version lock
Source Sass	Design system yang memerlukan konfigurasi mendalam	Kontrol tinggi; build lebih kompleks

Contoh 16. Package setup

```
npm install bootstrap@5.3.8

// entry.js – sesuaikan dengan bundler project
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap';
```

9.2 Mobile-first grid

Contoh 17. Grid responsif

```
<main class="container-xl py-4">
  <div class="row g-4">
    <section class="col-12 col-lg-8">
      <div class="row row-cols-1 row-cols-md-2 g-3">
        <div class="col"><article class="card h-100">...</article></div>
        <div class="col"><article class="card h-100">...</article></div>
      </div>
    </section>
    <aside class="col-12 col-lg-4">...</aside>
  </div>
</main>
```

Grid Bootstrap bersifat mobile-first, berbasis Flexbox, memiliki 12 kolom, dan menyediakan enam tier breakpoint pada dokumentasi 5.3. Class tanpa infix berlaku dari viewport terkecil; class dengan infix mulai aktif pada breakpoint terkait.

9.3 Reboot dan content

Reboot menetapkan baseline global pada elemen HTML. Karena dampaknya luas, integrasi Bootstrap ke aplikasi legacy harus diaudit. Gunakan utility dan komponen sesuai kontrak; pahami style typography, responsive image, table, dan figure sebelum menambah override.

Integrasi legacy: Jika tampilan berubah setelah Bootstrap dimuat, bandingkan computed style sebelum–sesudah dan identifikasi rule global. Jangan menetralkan seluruh framework dengan override besar.

PRAKTIK & EVIDENCE 9

Bangun layout dashboard Bootstrap dengan container, nested row/column, cards, responsive table, dan utilities. Uji tanpa CSS kustom terlebih dahulu.

Definition of done: Grid tidak memiliki row di luar container/column yang salah; gutter konsisten; tabel punya strategi mobile; CSS kustom hanya untuk gap yang benar-benar dibutuhkan; perubahan global terdokumentasi.

CHECKPOINT Bab 9

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa konsekuensi sistem 12 kolom?

Bagaimana class col-12 col-lg-8 berperilaku?

Mengapa Reboot perlu diperhatikan pada aplikasi legacy?

BAB 10

Bootstrap: Forms, Validation, dan Customization

Membangun form yang konsisten, accessible, dan dapat dikustomisasi tanpa merusak kontrak framework.

Klasifikasi	CORE
Fokus silabus	Forms and Form Components — controls, select/check/radio, layout, validation, floating labels, serta customization.
Project case	Form terlihat valid di browser tetapi server menerima data tidak sah, error tidak terhubung ke field, dan floating label membingungkan pengguna.
Bukti Intermediate	State normal/error/disabled/loading, keyboard test, server validation contract, dan customization rationale.
Best practice	Gunakan styling validasi untuk feedback; validasi dan otorisasi tetap wajib dilakukan di server.

TARGET KOMPETENSI

- Menggunakan form controls, checks/radios, layout, floating labels, dan feedback.
- Membedakan validasi visual dari validasi data/keamanan.
- Memilih CSS variables, Sass, atau override terukur untuk customization.

10.1 Struktur form

Contoh 18. Label, help, dan feedback

```
<form class="row g-3" novalidate>
  <div class="col-12 col-md-6">
    <label for="projectName" class="form-label">Nama project</label>
    <input id="projectName" name="projectName" class="form-control"
      required aria-describedby="projectNameHelp projectNameError">
    <div id="projectNameHelp" class="form-text">Gunakan nama resmi project.</div>
    <div id="projectNameError" class="invalid-feedback">Nama project wajib diisi.</div>
  </div>
  <div class="col-12">
    <button class="btn btn-primary" type="submit">Simpan</button>
  </div>
</form>
```

10.2 Validation

Class `:valid/:invalid` atau `.was-validated` membantu feedback visual setelah interaksi. Validasi browser dan client-side meningkatkan pengalaman pengguna, tetapi server tetap wajib memvalidasi serta mengotorisasi data. Jangan menyamakan tampilan hijau dengan data yang aman.

Aksesibilitas: Dokumentasi Bootstrap menyatakan custom validation tooltip belum sepenuhnya accessible. Utamakan feedback yang dapat diasosiasikan dengan field dan diuji menggunakan keyboard serta assistive technology.

10.3 Customization

Mekanisme	Gunakan ketika	Catatan
Utility/API	Kebutuhan sudah tersedia	Paling dekat dengan sistem Bootstrap
CSS variables	Perubahan token runtime/komponen	Scope dan fallback perlu jelas
Sass variables/maps	Build source dan konfigurasi sistem	Tetapkan sebelum import yang relevan
Override CSS	Kasus lokal yang terukur	Jaga specificity dan urutan import

Contoh 19. Scope CSS variables

```
.project-panel {
  --bs-card-border-color: #cbd5e1;
  --bs-card-border-radius: .875rem;
  --bs-card-cap-bg: #f8fafc;
}
```

PRAKTIK & EVIDENCE 10

Bangun form filter project dan form update status. Implementasikan state invalid setelah submit, help text, floating label hanya bila label tidak kehilangan konteks, serta customization scoped.

Definition of done: Setiap control memiliki label; feedback terhubung; keyboard flow logis; validasi server disebut sebagai requirement; customization tidak memakai selector ID panjang atau !important.

CHECKPOINT Bab 10

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa validasi client-side tidak cukup?

Kapan CSS variables lebih tepat daripada override global?

Apa risiko floating label tanpa pengujian konteks?

BAB 11

Debugging dan Keputusan Framework

Menemukan akar masalah secara sistematis dan memilih native CSS, Tailwind, atau Bootstrap berdasarkan kebutuhan project.

Klasifikasi	CORE
Fokus silabus	Responsive Design & Code Maintenance — debugging techniques, maintenance, serta pemilihan framework.
Project case	Tim menambah framework kedua untuk menyelesaikan issue styling, tetapi bundle, specificity, dan ownership justru memburuk.
Bukti Intermediate	Minimal reproduction, hypothesis log, decision matrix, regression suite, dan rollback plan.
Best practice	Pilih framework berdasarkan constraint project dan kemampuan tim; ukur total cost of change, bukan kecepatan demo awal.

TARGET KOMPETENSI

- Menjalankan siklus reproduce–inspect–isolate–fix–retest–document.
- Membedakan symptom dari root cause.
- Membuat keputusan framework dengan kriteria yang dapat dipertanggungjawabkan.

11.1 Alur debugging

Reproduce: tulis langkah, viewport, data, browser, dan hasil aktual.

Inspect: pilih elemen; periksa DOM, matched rules, computed value, box model, dan layout overlay.

Isolate: nonaktifkan satu declaration/rule atau buat minimal reproduction.

Fix: ubah pada layer/sumber yang memiliki ownership tepat.

Retest: uji kondisi awal, viewport tetangga, state keyboard, dan variasi data.

Document: catat akar masalah, perubahan, risiko, dan bukti regresi.

Symptom	Kemungkinan akar	Pemeriksaan
---------	------------------	-------------

Symptom	Kemungkinan akar	Pemeriksaan
Rule dicoret	Cascade/specificity/layer	Styles → rule pemenang dan source
Ukuran tidak sesuai	Box model/intrinsic min size	Computed + box model
Overlay tertutup	Stacking context ancestor	Peta layer/ancestor
Layout rusak mobile	Fixed size/min-content	Overflow source dan flex/grid overlay
Utility hilang	Build/source detection/import	Output CSS, command, class literal

11.2 Matriks keputusan

Kriteria	Native CSS	Tailwind CSS	Bootstrap
Desain unik	Sangat fleksibel	Fleksibel dengan token	Perlu customization
Kecepatan UI standar	Tergantung sistem internal	Cepat bagi tim utility-first	Sangat cepat dengan komponen
Markup	Class sesuai arsitektur	Utility padat	Class grid/komponen
Komponen JS	Dibangun sendiri	Tidak disediakan inti	Tersedia untuk pola tertentu
Legacy integration	Kontrol tinggi	Perlu pipeline dan scope	Audit global/Reboot
Skill tim	CSS reasoning kuat	Disiplin token/variant	Paham kontrak framework

Keputusan: Framework yang tepat adalah yang menurunkan total biaya delivery dan maintenance untuk konteks project—bukan yang memiliki class paling banyak atau paling populer.

11.3 Refactoring legacy CSS

- Kunci behavior dengan screenshot/test matrix.
- Petakan selector, duplikasi, specificity, dan dead rules.
- Tentukan boundary komponen serta token.

Refactor satu area kecil dan review diff.

Uji visual, responsive, keyboard, dan regresi.

Hapus rule lama setelah bukti pemakaian jelas.

PRAKTIK & EVIDENCE 11

Ambil satu bug CSS project dan isi debugging log lengkap. Lalu buat decision record satu halaman: native CSS, Tailwind, atau Bootstrap untuk fitur berikutnya.

Definition of done: Log memuat reproduksi, hipotesis, bukti, root cause, fix, dan regresi; keputusan framework memuat konteks, alternatif, trade-off, dan konsekuensi.

CHECKPOINT Bab 11

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa beda symptom dan root cause?

Mengapa minimal reproduction berguna?

Sebutkan tiga kriteria keputusan framework selain popularitas.

BAB 12

Implementasi Terpadu dan Mini-Project

Membuktikan kemampuan Intermediate melalui dashboard responsif, catatan debugging, dan penjelasan keputusan.

Klasifikasi	CORE — INTEGRATED VALIDATION
Fokus silabus	Seluruh kelompok CSS & Styling Frameworks.
Project case	Peserta membangun dashboard project yang responsif, dapat dibangun ulang, accessible, dan dapat dipertanggungjawabkan melalui evidence.
Bukti Intermediate	Repository, README, decision record, screenshot matriks uji, debugging log, checklist, dan technical defense.
Best practice	Nilai hasil dan proses. UI yang indah tetapi tidak reproducible, tidak teruji, atau tidak dapat dijelaskan belum memenuhi level Intermediate.

TARGET KOMPETENSI

- Mengintegrasikan layout, responsive, component state, framework, dan quality checks.
- Menyajikan evidence yang dapat diaudit reviewer.
- Menjelaskan trade-off dan rencana perbaikan.

12.1 Project Monitoring Dashboard

Bangun dashboard untuk memantau project, status, PIC, progres, risiko, dan aktivitas terbaru. Peserta boleh memilih native CSS, Tailwind CSS, atau Bootstrap setelah membuat decision record. Study case assessment final harus menggunakan soal dan dataset baru.

Minimum feature

- ☐ Header/navigation, page title, dan aksi utama.
- ☐ Empat metric cards dengan state normal/peringatan/kritis.
- ☐ Filter status, PIC, rentang tanggal, dan pencarian.

- ☐ Daftar/tabel project yang tetap usable pada mobile.
- ☐ Panel detail atau dialog update status.
- ☐ Empty, loading, error, disabled, hover, focus, dan validation state.
- ☐ Layout minimal tiga kondisi: mobile, medium, dan wide.

12.2 Constraint kualitas

Area	Constraint
Responsive	Tidak ada accidental horizontal overflow; breakpoint memiliki alasan konten
Accessibility	Semantic structure, label, focus-visible, keyboard flow, tidak mengandalkan warna
CSS	Tidak memakai !important sebagai default; specificity dan layer terkendali
Layering	Z-index memakai skala/token; modal/dropdown/sticky diuji
Framework	Versi/build tercatat; custom style dan override punya alasan
Evidence	Screenshot matriks uji, debugging log, dan technical reasoning tersedia

12.3 Tahapan kerja

Baca requirement dan petakan informasi serta state.

Buat semantic skeleton dan urutan keyboard.

Pilih model layout dan tulis decision record framework.

Bangun mobile baseline, lalu enhancement pada titik rusaknya konten.

Tambahkan token, komponen, state, dan motion terukur.

Uji data ekstrem, keyboard, zoom, viewport, serta overlay.

Perbaiki temuan dengan debugging log.

Siapkan demo dan penjelasan keputusan teknis.

12.4 Evidence package

- Repository dengan instruksi setup/build.

- README berisi keputusan framework dan struktur styling.
- Screenshot pada minimal lima kondisi uji.
- Satu debugging log lengkap.
- Checklist aksesibilitas/responsive.
- Catatan known limitations dan next improvement.

12.5 Pertanyaan review

Rule CSS mana yang paling rawan konflik dan bagaimana Anda mengendalikannya?

Mengapa breakpoint dipilih pada nilai tersebut?

Kapan Flexbox digunakan dan kapan Grid digunakan?

Apa stacking context pada dialog/dropdown?

Bagaimana pipeline framework memastikan style production tersedia?

Apa bukti UI tetap usable pada keyboard, zoom, data panjang, dan state error?

Jika project bertambah tiga tim, bagian styling mana yang paling mudah/mahal diubah?

Gate kelulusan: Project dinilai dari hasil dan proses. UI yang tampak baik tetapi tidak dapat dijelaskan, tidak dapat dibangun ulang, atau gagal pada kondisi uji belum memenuhi level Intermediate.

PRAKTIK & EVIDENCE 12

Selesaikan dashboard secara mandiri, lakukan peer review, perbaiki minimal dua temuan, lalu presentasikan keputusan serta bukti pengujiannya.

Definition of done: Semua feature dan constraint minimum terpenuhi; build dapat direproduksi; peserta dapat menjawab pertanyaan reviewer dengan merujuk source/DevTools/evidence; tidak ada blocker usability utama.

CHECKPOINT Bab 12

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa tiga bukti minimum bahwa dashboard responsif?

Mengapa technical reasoning menjadi bagian penilaian?

Apa beda mini-project pembelajaran dan study case assessment final?

Lampiran A1 — Traceability Detail 39 Topik Silabus

AUDIT CAKUPAN			
Tabel berikut memetakan seluruh 39 entri topik CSS & Styling Frameworks pada sheet Silabus Intermediate Programmer. Nomor mengikuti urutan topik pada domain CSS, bukan nomor soal.			
No.	Kategori	Topik silabus	Bab
1	CSS Properties	Knowledge more about CSS Properties	1–4, 6
2	CSS Properties	Difference about margin and padding	1
3	CSS Properties	Modification component with border, shadow etc	6
4	CSS Properties	Able to definition component layout, align and overflow	3
5	CSS Properties	Knowledge CSS Combinators	1
6	CSS Properties	How to used pseudo-class and pseudo-elements	1
7	CSS Properties	Declare CSS by attribute selector	1
8	CSS Properties	CSS counter & calculation	2
9	CSS Properties	CSS units such like px, em, %, vh, etc	2
10	CSS Properties	Best way to definition component z-indexes	4
11	CSS Responsive	Knowledge media queries	5
12	CSS Responsive	Definition media queries in script	5
13	CSS Flexboxes	Knowledge about CSS Flex layout	3
14	CSS Flexboxes	Definition media queries in script	5
15	Tailwind Introduction	Knowledge more about Core Concepts :	7–8
16	Tailwind Introduction	Hover, States, responsive design, reusing style, adding custom styles	5
17	Tailwind Introduction	How to responsive design	5
18	Tailwind Introduction	Tailwind adding custome styles	7–8
19	Tailwind Introduction	Definition functions & Directives	7–8
20	Styling Tailwind	Tailwind Grid & Flexbox	6–8
21	Styling Tailwind	Implementing transitions & Animations	6–8

No.	Kategori	Topik silabus	Bab
22	Responsive Design & Code Maintenance	Creating Responsive Design using Breakpoints in Tailwinds	5
23	Responsive Design & Code Maintenance	Debugging Techniques and code maintenance	5, 11
24	Introduction and Setup Bootstrap	Introduction to Bootstrap and its features.	9
25	Introduction and Setup Bootstrap	Setting up a basic project using Bootstrap CDN.	9
26	Introduction and Setup Bootstrap	Exploring and understanding the basic structure.	9
27	Introduction and Setup Bootstrap	Understanding Global Styles and Settings	9
28	Customizing Bootstrap	Utilizing CDN links for additional builds.	9
29	Customizing Bootstrap	Building with Bootstrap using source files via package manager.	9
30	Customizing Bootstrap	Using Bootstrap as a module with <code><script type="module"></code> .	9
31	Layout and Responsive Design	Understanding breakpoints and responsive design.	5
32	Layout and Responsive Design	Working with containers, grids, and columns.	9
33	Layout and Responsive Design	Exploring layout utilities, Z-index, and CSS Grid.	4
34	Content and Typography	Implementing Reboot for cross-browser rendering.	9
35	Content and Typography	Styling typography, images, and tables with Bootstrap.	9
36	Content and Typography	Working with figures and content-related features.	9
37	Forms and Form Components	Styling form controls and inputs.	10
38	Forms and Form Components	Implementing form components like select, checks, and radios.	10
39	Forms and Form Components	Working on form layouts, validation, and floating labels.	10

Lampiran A2 — Traceability Kelompok Silabus ke Modul

Kelompok silabus	Bab	Bukti
CSS properties, spacing, border, shadow	1, 2, 6	Computed style, token, component states
Selectors, pseudo, attribute, counters/calculations	1, 2	Selector refactor dan adaptive sizing
Display, alignment, overflow, Flexbox, Grid	3	Toolbar, metric grid, table region
Positioning dan z-index	4	Overlay map dan layer token
Media queries dan responsive maintenance	5, 11	Breakpoint evidence dan debugging log
Tailwind fundamentals hingga customization	7, 8	Build, variants, theme, component refactor
Bootstrap setup, layout, content, forms	9, 10	Grid dashboard, form, validation, customization
Integrated project validation	12	Dashboard dan evidence package

Lampiran B — Rubrik Self-Check Intermediate

Dimensi	Belum memenuhi	Memenuhi	Kuat
CSS reasoning	Trial-and-error dominan	Menjelaskan cascade/box/layout	Memprediksi dan membuktikan edge case
Responsive	Hanya dua screenshot	Mobile-first dan matriks uji	Data/zoom/state ekstrem teruji
Framework	Menyalin class	Build dan kontrak dipahami	Customization dan trade-off terukur
Accessibility	Focus/label terlewat	Keyboard, label, state jelas	A11y menjadi acceptance criteria
Debugging	Mengubah banyak hal sekaligus	Root cause dan retest tercatat	Minimal reproduction dan regresi kuat
Maintainability	Duplikasi/specificity tinggi	Token dan boundary konsisten	Refactor bertahap dengan evidence

Interpretasi: Peserta siap untuk assessment project bila seluruh dimensi minimal berada pada kolom Memenuhi dan tidak ada blocker accessibility, responsive, atau build reproducibility.

Lampiran C — Kunci Quick Check

Nomor	Jawaban inti
1.1	Border-box memasukkan padding dan border ke dalam width/height yang ditetapkan.
1.2	Selector makin sulit dioverride dan meningkatkan coupling/perang specificity.
1.3	Saat state sudah direpresentasikan attribute DOM/ARIA/data dan perlu diseleksi langsung.
2.1	Saat ukuran komponen perlu mengikuti skala font lokal elemen.
2.2	Menjaga fluida tetapi tetap dalam batas keterbacaan/layout.
2.3	Intrinsic min-content child dapat memaksa track melebar; nilai 0 mengizinkan shrink.
3.1	Flexbox dominan satu axis; Grid mengendalikan baris dan kolom bersama.
3.2	Track boleh menyusut sampai 0 sehingga konten tidak memaksa grid melebar.
3.3	Item menolak shrink dan dapat menyebabkan overflow.
4.1	Nilai hanya dibandingkan dalam stacking context yang relevan; ancestor dapat membatasi.
4.2	Absolute diposisikan terhadap containing block terdekat yang memenuhi aturan.
4.3	Periksa inset, scroll container, overflow ancestor, ruang scroll, dan context.
5.1	Base rule untuk kecil; enhancement min-width diletakkan setelahnya.
5.2	Karena perubahan harus terjadi saat layout/konten membutuhkan ruang.
5.3	Ia menyembunyikan symptom dan dapat memotong informasi/focus.
6.1	Token global mewakili keputusan sistem; nilai lokal hanya relevan pada konteks terbatas.
6.2	Properti tak terduga ikut dianimasikan dan dapat memicu motion/performa yang tidak perlu.
6.3	Motion dikurangi/dinonaktifkan tanpa menghilangkan informasi atau fungsi.

Nomor	Jawaban inti
7.1	Mengimpor Tailwind pada file input agar pipeline menghasilkan utility yang digunakan.
7.2	Utility tetap memetakan properti, cascade, layout, dan responsive CSS.
7.3	Build scanner mungkin tidak menemukan class sehingga CSS tidak dihasilkan.
8.1	Saat pola stabil/berulang atau selector kompleks memberi manfaat nyata.
8.2	Nilai desain tersebar dan tidak lagi mengikuti theme bersama.
8.3	Konfirmasi output CSS, import, command build, dan class literal pada source.
9.1	Lebar kolom dinyatakan sebagai bagian dari 12 dan dapat dikombinasikan per breakpoint.
9.2	Penuh sampai lg, lalu 8/12 pada lg ke atas.
9.3	Baseline global dapat mengubah style elemen yang sudah ada.
10.1	Input dapat dimanipulasi; server harus memvalidasi dan mengotorisasi ulang.
10.2	Saat token komponen/runtime dapat diubah melalui variable dengan scope jelas.
10.3	Label dapat kehilangan kejelasan, terutama saat nilai terisi/placeholder dipakai salah.
11.1	Symptom adalah hasil terlihat; root cause adalah mekanisme yang menimbulkannya.
11.2	Mengurangi variabel agar penyebab dapat dibuktikan dan dibagikan.
11.3	Misalnya desain, skill tim, pipeline, legacy, komponen JS, maintenance, dan performance.
12.1	Matriks viewport, bukti tanpa overflow, dan uji zoom/data/state.
12.2	Level Intermediate menilai kemampuan mempertanggungjawabkan dan memelihara solusi.
12.3	Mini-project membangun kemampuan; assessment memakai study case baru untuk validasi independen.

Lampiran D — Template Debugging Log

Field	Isi yang harus dicatat
Issue	Ringkasan masalah dan dampak pengguna
Environment	Browser, viewport, commit/build, data/state

Field	Isi yang harus dicatat
Reproduction	Langkah minimum yang konsisten
Expected/actual	Perbedaan hasil yang dapat diamati
Hypotheses	Dugaan terurut sebelum perubahan
Evidence	Matched rule, computed value, box/layout, screenshot
Root cause	Mekanisme teknis penyebab
Fix	Perubahan dan ownership layer/source
Regression	Kondisi tetangga yang diuji ulang
Follow-up	Debt atau peningkatan lanjutan

Lampiran E — Glosarium

Istilah	Definisi ringkas
Cascade	Algoritme yang menentukan declaration CSS yang berlaku.
Specificity	Bobot selector dalam satu tahap cascade.
Computed value	Nilai property setelah cascade, inheritance, dan komputasi awal.
Containing block	Kotak referensi untuk ukuran/posisi elemen tertentu.
Intrinsic size	Ukuran yang berasal dari konten/asset itu sendiri.
Stacking context	Konteks tiga dimensi tempat layer child dibandingkan.
Breakpoint	Kondisi media saat layout membutuhkan perubahan.
Design token	Nama bermakna untuk keputusan desain berulang.
Utility-first	Pendekatan membangun UI dari class tujuan tunggal yang dapat dikomposisikan.
Reboot	Baseline global Bootstrap untuk konsistensi elemen HTML.
Regression	Kerusakan baru pada behavior yang sebelumnya bekerja.

Istilah	Definisi ringkas
Technical reasoning	Penjelasan berbasis requirement, mekanisme, bukti, dan trade-off.

Lampiran F — Referensi Resmi

Akses dokumentasi sesuai versi package yang digunakan project. Tanggal akses: 20 Agustus 2026.

- [MDN — Introduction to the CSS box model](#)
- [MDN — Specificity](#)
- [MDN — Flexbox](#)
- [MDN — Grid layout](#)
- [MDN — Stacking context](#)
- [MDN — Using media queries](#)
- [Tailwind CSS — Responsive design](#)
- [Tailwind CSS — Theme variables](#)
- [Tailwind CSS — Functions and directives](#)
- [Tailwind CSS — Adding custom styles](#)
- [Tailwind CSS — CLI installation](#)
- [Tailwind CSS — PostCSS installation](#)
- [Bootstrap 5.3 — Grid system](#)
- [Bootstrap 5.3 — Breakpoints](#)
- [Bootstrap 5.3 — Forms overview](#)
- [Bootstrap 5.3 — Validation](#)
- [Bootstrap 5.3 — Customize](#)

Penutup

Standar Intermediate: Peserta yang kompeten mampu memprediksi, membuktikan, memperbaiki, dan menjelaskan. Framework mempercepat implementasi, tetapi kualitas akhir tetap ditentukan oleh reasoning CSS, pengujian, dan disiplin maintenance.